VoiceBridge — Complete Project Roadmap

PHASE 1 — What i Already Have (Done)

- STT (Speech-to-Text) using Web Speech API
- TTS (Text-to-Speech) using Speech Synthesis API
- Basic AI Chatbot with fixed Q&A
- Modal controls
- Tab switching (STT / TTS)
- Report form with validation
- Toast notifications

🔧 **PHASE 2 — Strengthen the Foundation**

1. Multi-language Support

- Add a language selector dropdown
- Pass selected language to recognition.lang for STT
- Pass selected language to SpeechSynthesisUtterance.lang for TTS

javascript

```
recognition.lang = 'ka-IN'; // Hindi example
utterance.lang = 'hi-IN';
```

2. STT Improvements

- Show interim results in real time while speaking
- Add punctuation support
- Add a "Copy to Clipboard" button on STT output

3. TTS Improvements

- Add voice selector (male/female)
- Add speed and pitch controls
- Add pause/resume/stop buttons

📞 PHASE 3 — Real-Time Call Captioning (Your Main Goal)

This is the core Nagish-like feature. Here's how to build it:

Step 1 — Basic Audio Call Between Two Users

- Use **WebRTC** for peer-to-peer audio/video calls
- Use **PeerJS** library (simplifies WebRTC)

javascript

```
// Install or use CDN
// https://cdnjs.cloudflare.com/ajax/libs/peerjs/1.4.7/peerjs.min.js


const peer = new Peer();
peer.on('open', (id) => {
  console.log('My peer ID: ' + id);
});
```

Step 2 — Capture Audio From the Call

- Capture the microphone stream during the call
- Feed it into Web Speech API for live captioning

javascript

```
const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
recognition.start(); // runs alongside the call
```

Step 3 — Show Live Captions on Screen

- Display real-time transcription as the person speaks
- Show captions at the bottom of the call screen like subtitles

Step 4 — Send Captions to the Other Person

- Use **WebRTC DataChannel** to send caption text in real time
- Other person sees your speech as text on their screen instantly

---

🌐 PHASE 4 — Backend & User Accounts

- Use **Node.js + Express** for the backend server

- Use **Socket.io** for real-time signaling between users
- Use **MongoDB** to store user profiles and call history
- Add login / signup with **JWT authentication**

Basic server setup:

javascript

```
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = new Server(server);

io.on('connection', (socket) => {
  console.log('User connected:', socket.id);

  socket.on('caption', (text) => {
    socket.broadcast.emit('receive-caption', text);
  });
});

server.listen(3000);
```

🤖 PHASE 5 — Upgrade the Chatbot

- Replace fixed Q&A with **Claude API / OpenAI API** for smart responses
- Add chatbot ability to **help during calls** (e.g. suggest phrases)
- Add **sign language GIF suggestions** for deaf users

📱 PHASE 6 — Mobile & Accessibility

- Make the UI fully **responsive** for mobile
- Add **PWA (Progressive Web App)** support so users can install it
- Follow **WCAG accessibility guidelines**
- Add keyboard navigation support
- Add high contrast mode for visually impaired users

☁️ PHASE 7 — Deployment

- Host frontend on **Netlify** or **Vercel** (free)
- Host backend on **Render** or **Railway** (free tier)
- Use **MongoDB Atlas** for free cloud database
- Buy a domain and add **SSL certificate**

🧪 PHASE 8 — Testing & Launch

- Test with real deaf/hard-of-hearing users
- Fix latency issues in captions
- Optimize for slow internet connections (important for India)
- Add feedback form (you already have report form ✅)
- Launch and share with accessibility communities in India

📊 Your Progress Summary

Phase	Status
Phase 1 — Basic STT/TTS/Chatbot	✅ Done
Phase 2 — Strengthen Features	🔧 Next Step
Phase 3 — Real-Time Call Captioning	🎯 Main Goal
Phase 4 — Backend & Auth	⏳ Later

Phase	Status
Phase 5 — Smart Chatbot	⏳ Later
Phase 6 — Mobile & Accessibility	⏳ Later
Phase 7 — Deployment	⏳ Later
Phase 8 — Testing & Launch	⏳ Later